

Stack and Heap for Point Addition

When we write `Point p1 = new Point(2, 3);`, two different things happen in memory. Java doesn't put the actual object where the variable is. Instead, it creates the variable `p1` on the **stack**, and the real object (with `x = 2` and `y = 3`) gets created separately on the **heap**.

So `p1` doesn't hold the actual values. It just holds the **address** of where the object is stored on the heap. I like to think of it like `p1` is just a piece of paper with directions written on it, and the real thing you're looking for is somewhere else (on the heap).

The same thing happens for `p2`. It's a separate variable on the stack, pointing to its own separate object on the heap.

Now when we call `p1.add(p2)`, Java runs the `add()` method. Inside that method, it takes the `x` and `y` values from both points, adds them, and then creates a **brand new** `Point` object with the new values. This new object also goes on the heap. Then Java returns the address of this new object, and we store that address in a new variable called `sum`.

The important part here is that `p1` and `p2` do **not** change. They are still pointing to their original objects on the heap with the original values. The `add()` method didn't touch them — it just built a completely new object and gave us its address back.

This is also why the **copy constructor** matters. If I write `Point p3 = new Point(p1);`, Java creates a whole new object on the heap with the same `x` and `y` values as `p1`. But if I had written `Point p3 = p1;` instead (without `new`), then `p3` would just be another variable pointing to the **same** object as `p1` — not a new copy. That means if `p1`'s values ever changed, `p3` would "see" that change too, because they'd both be looking at the same thing on the heap. The copy constructor avoids that by making sure `p3` gets its own separate object.

So basically: **stack = where variables live, heap = where the actual objects live, and variables are just addresses pointing to the heap.**

